

UNIT-II

Basic SQL:

15 Hours

Basic SQL: SQL Data Definition and Data Types, Specifying Constraints in SQL, Basic retrieval queries in SQL, Insert, Delete and Update Statements in SQL,

More SQL: Complex Queries, Views, and Schema Modification: More complex SQL retrieval

queries, Specifying constraints as assertions and Actions as Triggers, Views in SQL, Schema Change Statements in SQL.

Basics of Functional Dependencies and Normalization for Relational Databases: Informal design guidelines for relation schemas, Functional Dependencies, Normal Forms Based on Primary Keys, general definitions of Second and Third Normal Forms, Boyce-Codd Normal Form.

Relational Database Design Algorithms and Further Dependencies: Inference Rules, Equivalence, and Minimal cover, Properties of Relational Decompositions, Algorithms for Relational Database Schema.

(T1: 6.1-6.4 ,7.1-7.4, 14.1-14.5, 15.1, 15.2,15.3)

Chapter 6

Basic SQL

INTRODUCTION

- The name SQL is expanded as **Structured Query Language**.
- Originally, SQL was called **SEQUEL (Structured English QUery Language)**
- A high-level declarative language interface.
- Includes some features from relational algebra

SQL DATA DEFINITION AND DATA TYPES

- In SQL terminology ,
 - A relation is called a **table**
 - A tuple is called a **row**
 - An attribute is called a **column**
- **CREATE** statement : The main SQL command for data definition.

It can used to create

- **schemas**
- **tables**
- **domains**
- **views**
- **Assertion**
- **triggers**

- A schema is created via the **CREATE SCHEMA** statement, which can include all the schema elements' definitions.

Example:

The following statement creates a schema called COMPANY, owned by the user with authorization identifier JSMITH

CREATE SCHEMA COMPANY AUTHORIZATION JSMITH;

*In general , not all users are authorized to create schema and schema elements .
The privilege for these must be given by Database Administrator (DBA)*

SQL catalog

A named collection of schemas in an SQL environment.

- A catalog always contains a special schema called **INFORMATION_SCHEMA**
- **INFORMATION_SCHEMA** provides information on all the element descriptors of all the schemas in the catalog to authorized users.

SQL Data Definition : The CREATE TABLE Command

- Used to specify a new table by giving it a name and specifying its attributes and constraints
 - The **attributes** are specified first, and each attribute is given a name
 - Followed by a **data type** to specify its domain of values.
 - Followed by any attribute constraints such as NOT NULL.
 - The key, entity integrity, and referential integrity constraints can be specified—within the CREATE TABLE statement—after the attributes are declared.
- **Syntax :**

CREATE TABLE COMPANY.EMPLOYEE...

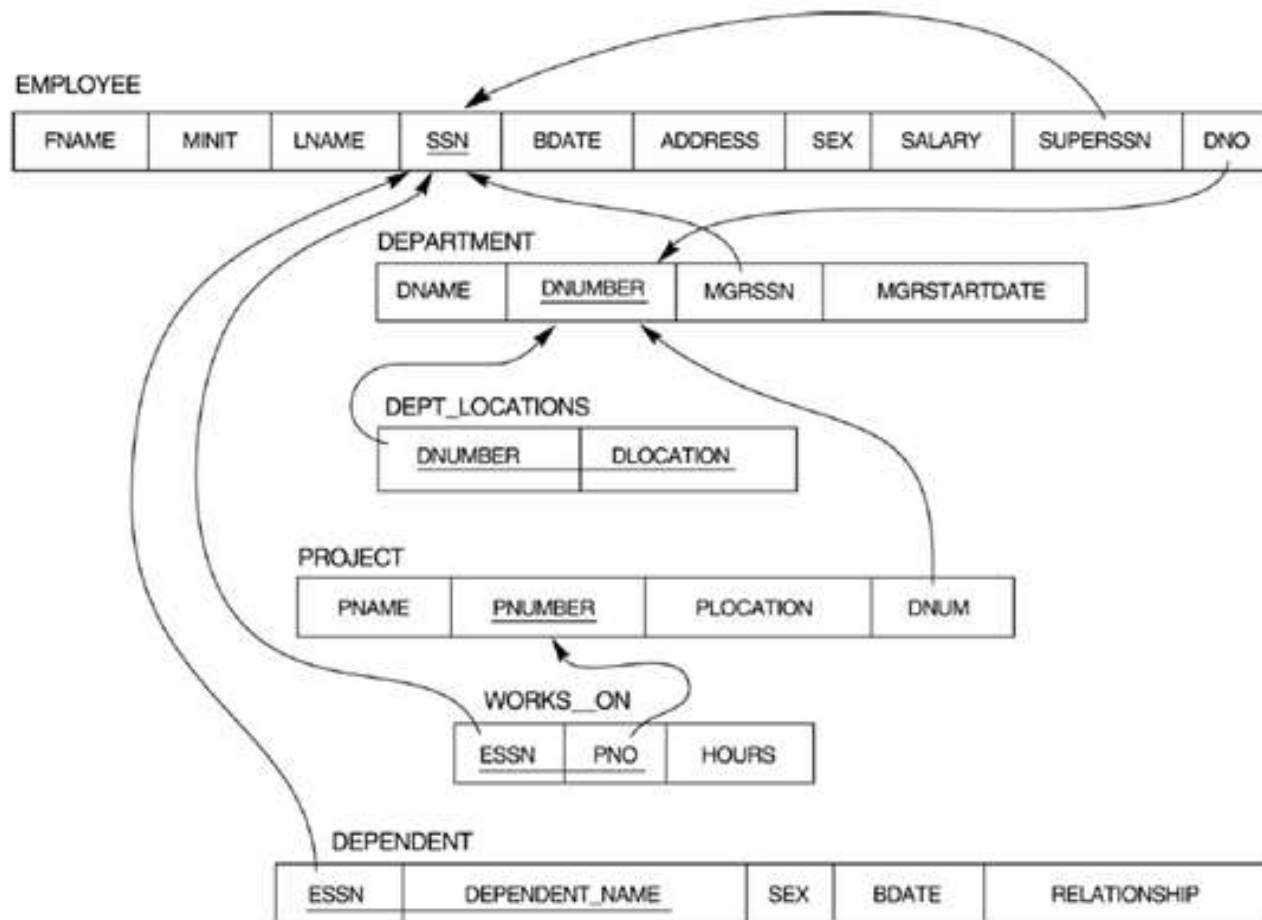
OR

CREATE TABLE EMPLOYEE ...



- The relations declared through CREATE TABLE statements are called **base tables** (or base relations)
- The relations declared through CREATE VIEW statements are called **virtual tables** (or virtual relations)
- In SQL the attributes in a base relation are considered to be *ordered in the sequence* in which they are specified in the CREATE table statement.
- Rows are not considered to be ordered within a relation

Example : Company Database Schema



SQL Data Definition : The CREATE TABLE Command

Example : Company Database

CREATE TABLE EMPLOYEE

(FNAME	VARCHAR(15)	NOT NULL ,
MINIT	CHAR ,	
LNAME	VARCHAR(15)	NOT NULL ,
SSN	CHAR(9)	NOT NULL ,
BDATE	DATE	
ADDRESS	VARCHAR(30) ,	
SEX	CHAR ,	
SALARY	DECIMAL(10,2) ,	
SUPERSSN	CHAR(9) ,	
DNO	INT	NOT NULL ,

PRIMARY KEY (SSN) ,

FOREIGN KEY (SUPERSSN) **REFERENCES** EMPLOYEE(SSN) ,

FOREIGN KEY (DNO) **REFERENCES** DEPARTMENT(DNUMBER)) ;

CREATE TABLE DEPARTMENT

(DNAME	VARCHAR(15)	NOT NULL ,
DNUMBER	INT	NOT NULL ,
MGRSSN	CHAR(9)	NOT NULL ,
MGRSTARTDATE	DATE ,	

PRIMARY KEY (DNUMBER) ,

UNIQUE (DNAME) ,

FOREIGN KEY (MGRSSN) **REFERENCES** EMPLOYEE(SSN)) ;

CREATE TABLE DEPT_LOCATIONS

(DNUMBER	INT	NOT NULL ,
DLOCATION	VARCHAR(15)	NOT NULL ,

PRIMARY KEY (DNUMBER, DLOCATION) ,

FOREIGN KEY (DNUMBER) **REFERENCES** DEPARTMENT(DNUMBER)) ;

CREATE TABLE PROJECT

(PNAME VARCHAR(15) **NOT NULL** ,
 PNUMBER INT **NOT NULL** ,
 PLOCATION VARCHAR(15) ,
 DNUM INT **NOT NULL** ,
PRIMARY KEY (PNUMBER) ,
UNIQUE (PNAME) ,
FOREIGN KEY (DNUM) **REFERENCES** DEPARTMENT(DNUMBER)) ;

CREATE TABLE WORKS_ON

(ESSN CHAR(9) **NOT NULL** ,
 PNO INT **NOT NULL** ,
 HOURS DECIMAL(3,1) **NOT NULL** ,
PRIMARY KEY (ESSN, PNO) ,
FOREIGN KEY (ESSN) **REFERENCES** EMPLOYEE(SSN) ,
FOREIGN KEY (PNO) **REFERENCES** PROJECT(PNUMBER)) ;

CREATE TABLE DEPENDENT

(ESSN CHAR(9) **NOT NULL** ,
 DEPENDENT_NAME VARCHAR(15) **NOT NULL** ,
 SEX CHAR ,
 BDATE DATE ,
 RELATIONSHIP VARCHAR(8) ,
PRIMARY KEY (ESSN, DEPENDENT_NAME) ,
FOREIGN KEY (ESSN) **REFERENCES** EMPLOYEE(SSN)) ;

Attribute Data Types and Domains in SQL

1. **Numeric** : Integer, Float, Decimals
2. **Character-string**: char, varchar
3. **Bit-string**: BLOB
4. **Boolean**: True / False
5. **DATE**: date
6. **Timestamp**: Date and Time fields

- **Numeric data types**

- Integer numbers of various sizes
 INTEGER or **INT** and **SMALLINT**
- Floating points of various precision
 FLOAT or **REAL**
 DOUBLE PRECISION
- Formatted numbers can be declared by using
 - **DECIMAL** (i , j) or **DEC**(i , j) or **NUMERIC**(i , j) where i is the precision and j is the scale
 - The default scale is zero and default for precision is implementation defined.

- **Character-string → 2 types**

- 1. Fixed-length**

`CHAR(n)` or `CHARACTER(n)`, where *n* is the number of characters.

- 2. Varying-length**

`VARCHAR(n)` or `CHAR VARYING(n)` or `CHARACTER VARYING(n)`,
where *n* is the maximum number of characters.

- Literal string value is placed between single quotation marks and it is case sensitive.

Example `'SMITH'` and `'smith'`

- **Bit-string → 2 types**

- 1. Fixed length**

`BIT(n)`, where *n* is the maximum number of bits.

- 2. Varying length**

`BIT VARYING(n)`, where *n* is the maximum number of bits.

The default for *n*, the length of a character string or bit string, is one.

- Literal bit strings are placed between single quotes but precede by a B ;

Example `B'10101'`

- **Boolean**

- **Traditional Values: TRUE or FALSE**
- **Third possible values : UNKNOWN**

TABLE 8.1 LOGICAL CONNECTIVES IN THREE-VALUED LOGIC

AND	TRUE	FALSE	UNKNOWN
TRUE	TRUE	FALSE	UNKNOWN
FALSE	FALSE	FALSE	FALSE
UNKNOWN	UNKNOWN	FALSE	UNKNOWN
OR	TRUE	FALSE	UNKNOWN
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	UNKNOWN
UNKNOWN	TRUE	UNKNOWN	UNKNOWN
NOT			
TRUE	FALSE		
FALSE	TRUE		
UNKNOWN	UNKNOWN		

- **Date and Time**

- The **DATE** data type has ten positions

Typically form **YYYY-MM-DD**.

- The **TIME** data type has at least eight positions

Typically form **HH:MM:SS**.

- Literal values are represented by single quoted strings preceded by DATE or TIME

DATE'2002-09-27' or **TIME**'09:12:47'

- The **< (less than)** comparison can be used with dates or times.

- **Timestamp**

- A timestamp data type (**TIMESTAMP**) includes both the DATE and TIME fields, plus a maximum number of six positions for decimal fractions of seconds and an optional WITH TIME ZONE qualifier
- Literal values are specified as follows:

TIMESTAMP'2002 -09-27 09:12:47 648302'

(Minimum one space between date and time)

- **Interval**

- **INTERVAL** data type specifies an **interval**—a *relative value* that can be used to increment or decrement an absolute value of a date, time, or timestamp.

SPECIFYING CONSTRAINTS IN SQL

1. Not Null constraints :

- This should always be specified for the primary key attributes of each relation,
- Also for any other attributes whose values are required not to be NULL.

Example : DNAME VARCHAR(20) NOT NULL

2. DEFAULT Constraint:

- To define a default value for an attribute

Example : DNO INT NOT NULL DEFAULT 1

3. CHECK Constraint :

- To restrict the attribute or domain values.

Example :

DNUMBER INT NOT NULL CHECK (DNUMBER > 0 AND DNUMBER < 21)

Example:

Mgr_ssn CHAR(9) NOT NULL DEFAULT '888665555'

- Another type of constraint can restrict attribute or domain values using the CHECK clause following an attribute or domain definition.

Example:

Dnumber INT NOT NULL CHECK (Dnumber > 0 AND Dnumber < 21);

- We can create your own domain as follows:

Example:

```
CREATE DOMAIN D_NUM AS INTEGER CHECK  
(D_NUM > 0 AND D_NUM < 21);
```

- Then use the created domain D_NUM as the attribute type for all attributes that refer to department numbers.
- **Eg:** Dnumber of DEPARTMENT, Dnum of PROJECT, Dno of EMPLOYEE and so on.

■ Specifying Key and Referential Integrity Constraints

Primary key specifies one or more attributes that make up the primary key of a relation.

PRIMARYKEY(DNUMBER,DLOCATION)

The PRIMARY KEY constraint uniquely identifies each record in a database table.

Primary keys must contain unique values, and cannot contain NULL values. A table can have only one primary key, which may consist of single or multiple fields.

Dnumber INT PRIMARY KEY;

- The **UNIQUE** clause specifies alternate (secondary) keys, The UNIQUE constraint ensures that all values in a column are different. Both the UNIQUE and PRIMARY KEY constraints are similar.
- A table can have many UNIQUE constraints , but only one PRIMARY KEY constraint per table.

Dname VARCHAR(15) **UNIQUE**;

- A FOREIGN KEY is a key used to link two tables together.
- A FOREIGN KEY is a field (or collection of fields) in one table that refers to the PRIMARY KEY in another table.

- A referential integrity constraint can be violated when tuples are inserted or deleted, or when a foreign key or primary key attribute value is modified.

FOREIGN KEY (DNO) REFERENCES DEPARTMENT(DNUMBER)

```
CREATE TABLE DEPARTMENT
(
  Dname VARCHAR(15) NOT NULL,
  Dnumber INT NOT NULL,
  Mgr_ssn CHAR(9) NOT NULL,
  Mgr_start_date DATE,
  PRIMARY KEY (Dnumber),
  UNIQUE (Dname),
  FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn)
);
```

Options to avoid violations

1. The options include SET NULL, CASCADE, and SET DEFAULT.
2. An option must be qualified with either ON DELETE or ON UPDATE.
3. The action for CASCADE ON DELETE is to delete all the referencing tuples,
4. The action for CASCADE ON UPDATE is to change the value of the foreign key to the updated (new) primary key value for all referencing tuples.

Example:

**FOREIGN KEY (DNO) REFERENCES DEPARTMENT(DNUMBER)
ON DELETE SET DEFAULT ON UPDATE CASCADE.**

GIVING NAMES TO THE CONSTRAINTS:

- **Constraints** may be given a name, following the keyword **CONSTRAINT**.

Example :

CONSTRAINT *DETPK* **PRIMARY KEY**(**DNUMBER**)

- The names all the constraints must be unique within a schema.
- Constraints can be dropped or replaced with another constraint.
- Giving names to constraints is optional.

Specifying Constraints on Tuples Using CHECK

- These are called tuple based constraints which are specified at the end of a CREATE TABLE statement.

Example : `CHECK ( DEPT_CREATE_DATE <MGRSTARTDATE);`

Because a manager can start managing a department only after it is created

- Logically, a manager cannot start working in a department before the department exists.
- This CHECK constraint ensures that such incorrect data is not entered into the table.

The **DROP** Command

- **To drop**
 - schema
 - tables
 - domains
 - constraints.
- **DROP has two options : CASCADE & RESTRICT**

Example : DROP SCHEMA COMPANY CASCADE;

- **CASCADE** option , **when dropping a schema** , will remove database schema and all its tables, domains, and other elements.
- **If RESTRICT** option is chosen ,then schema is dropped only if it has no elements in it; otherwise, the DROP command will not be executed

The **ALTER TABLE** Command

- To change the definition of a base table
- **ALTER TABLE** is used to
 1. add a column (attribute)
 2. drop a column (attribute)
 3. change column definition
 4. add or dropping table constraints.

Examples:

1. **ALTER TABLE EMPLOYEE ADD JOB VARCHAR(12);**
2. **ALTER TABLE EMPLOYEE DROP ADDRESS CASCADE;**
3. **ALTER TABLE DEPARTMENT ALTER MGRSSN DROP DEFAULT;**
4. **ALTER TABLE DEPARTMENT ALTER MGRSSN SET DEFAULT "333445555";**
5. **ALTER TABLE EMPLOYEE DROP CONSTRAINT EMPSUPERFK CASCADE;**

BASIC RETRIEVAL QUERIES IN SQL

- The **SELECT-FROM-WHERE** structure of Basic SQL Queries

```
SELECT    <attribute list>  
           FROM    <table list>  
           WHERE   <condition>;
```

- **<attribute list>** is a list of attribute names whose values are to be retrieved by the query
- **<table list>** is a list of the relation names required to process the query
- **<condition>** is a conditional expression that identifies the tuples to be retrieved by the query
- Operators like =, <=, >, >=, <> can be used in the condition.

- Query1
- Retrieve the birth date and address of the employee whose name is 'John B Smith'

```
SELECT BDATE, ADDRESS  
FROM EMPLOYEE  
WHERE FNAME='John' AND MINIT='B' AND LNAME='Smith'
```

* the result of the query *may contain* duplicate tuples

• **Equivalent relational algebra expression** (duplicates would not be eliminated)

$$\pi_{\text{BDATE}, \text{ADDRESS}} (\sigma_{\text{FNAME}='John' \text{ AND } \text{MINIT}='B' \text{ AND } \text{LNAME}='Smith'} (\text{EMPLOYEE}))$$

- **Query 2:**
- **Retrieve the name and address of all employees who work for the 'Research' department.**

```
SELECT  FNAME, LNAME, ADDRESS  
FROM    EMPLOYEE, DEPARTMENT  
WHERE    DNUMBER=DNO AND DNAME='Research'
```

- Similar to a SELECT-PROJECT-JOIN sequence of relational algebra operations
- (DNAME='Research') is a *selection condition* (corresponds to a SELECT operation in relational algebra)
- (DNUMBER=DNO) is a *join condition* (corresponds to a JOIN operation in relational algebra)

```
SELECT FNAME, LNAME, ADDRESS  
FROM EMPLOYEE  
WHERE DNO <> (  
    SELECT DNUMBER  
    FROM DEPARTMENT  
    WHERE DNAME = 'Research'  
);
```

**To retrieve employees
who do not work in
research dept**

- Query 3:
- For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birth date.

```
SELECT PNUMBER, DNUM, LNAME, BDATE, ADDRESS  
FROM PROJECT, DEPARTMENT, EMPLOYEE  
WHERE DNUM=DNUMBER AND MGRSSN=SSN AND PLOCATION='Stafford'
```

In Q2, there are *two* join conditions and one select condition

- The join condition **DNUM=DNUMBER** relates a project to its controlling department.
- The join condition **MGRSSN=SSN** relates the department to the employee who manages that department.
- **PLOCATION='Stafford'** is selection condition for Project table

AMBIGUOUS ATTRIBUTE NAMES, ALIASING, RENAMING AND TUPLE VARIABLES

Ambiguous attribute names

- A query may refer to two or more attributes with the same name
- In such cases, we must *qualify* the attribute name with the relation name
- This is done by *prefixing* the relation name to the attribute name by using *dot(.)* notation to prevent ambiguity.

Example:

Suppose that LNAME and DNO attributes of EMPLOYEE table were called NAME and DNUMBER, and DNAME attribute of the DEPARTMENT was also called NAME. then ***Query1 must be restated as follows.***

```
SELECT  FNAME, EMPLOYEE.NAME, ADDRESS  
FROM    EMPLOYEE, DEPARTMENT  
WHERE   DEPARTMENT.NAME='Research' AND  
EMPLOYEE.DNUMBER=DEPARTMENT. DNUMBER
```

Aliasing and Tuple Variables

- Ambiguity also arises if some queries need to refer to the same relation twice
 - In this case, *aliases* are given to the relation name using *AS*
- **Query :**
 - **For each employee, retrieve the employee's name, and the name of his or her immediate supervisor.**

SELECT E.FNAME, E.LNAME, S.FNAME, S.LNAME

FROM EMPLOYEE AS E, EMPLOYEE AS S

WHERE E.SUPERSSN=S.SSN

- the alternative relation names **E** and **S** are called *aliases* or *tuple variables* for the **EMPLOYEE** relation
- We can think of **E** and **S** as two *different copies* of **EMPLOYEE**;
- **E** represents employees in role of *supervisees* and **S** represents employees in role of *supervisors*
- *Use of AS is optional.*

E.g. : **Use EMPLOYEE E, instead of EMPLOYEE AS E**

UNSPECIFIED WHERE-CLAUSE

- A *missing WHERE-clause* indicates no condition; hence, all tuples of the relations in the FROM-clause are selected
 - This is equivalent to the condition WHERE TRUE
- **Query : Retrieve the SSN values for all employees.**

```
SELECT  SSN
FROM    EMPLOYEE
```

- If more than one relation is specified in the FROM-clause *and* there is no join condition, then the *CARTESIAN PRODUCT* of tuples is selected

USE OF *

- To retrieve all the attribute values of the selected tuples, a * is used, which stands for *all the attributes*

- **Eg:**

Query: SELECT *
 FROM EMPLOYEE
 WHERE DNO=5

Query: SELECT *
 FROM EMPLOYEE, DEPARTMENT
 WHERE DNAME='Research' AND
 DNO=DNUMBER

- SQL doesn't remove duplicate tuples automatically in the results of queries.
- To eliminate duplicate tuples in a query result, the keyword **DISTINCT** is used
- **Eg:** The result of Q may have duplicate SALARY values whereas Q11A does not have any duplicate values

Q: SELECT SALARY FROM EMPLOYEE

QA: SELECT DISTINCT SALARY FROM EMPLOYEE

Result of Q

<u>SALARY</u>
30000
40000
25000
43000
38000
25000
25000
55000

Result of QA

<u>SALARY</u>
30000
40000
25000
43000
38000
55000

- SQL has directly incorporated some set operations.
- There is a union operation (UNION), and in *some versions* of SQL there are set difference (EXCEPT) and intersection (INTERSECT) operations.
- The resulting relations of these set operations are sets of tuples; *duplicate tuples are eliminated from the result.*
- The set operations apply only to *union compatible relations*; the two relations **must have the same attributes and the attributes must appear in the same order**

Set Theoretic Operations

EMPLOYEE	FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO
	John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
	Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
	Alicia	J	Zelaya	999887777	1968-07-19	3321 Castle, Spring, TX	F	25000	987654321	4
	Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
	Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
	Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
	Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
	James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	null	1

DEPT_LOCATIONS					DNUMBER	DLOCATION
					1	Houston
					4	Stafford
					5	Bellaire
					5	Sugarland
					5	Houston

DEPARTMENT	DNAME	DNUMBER	MGRSSN	MGRSTARTDATE
	Research	5	333445555	1988-05-22
	Administration	4	987654321	1995-01-01
	Headquarters	1	888665555	1981-06-19

WORKS_ON	ESSN	PNQ	HOURS
	123456789	1	32.5
	123456789	2	7.5
	666884444	3	40.0
	453453453	1	20.0
	453453453	2	20.0
	333445555	2	10.0
	333445555	3	10.0
	333445555	10	10.0
	333445555	20	10.0
	999887777	30	30.0
	999887777	10	10.0
	987987987	10	35.0
	987987987	30	5.0
	987654321	30	20.0
	987654321	20	15.0
	888665555	20	null

PROJECT	PNAME	PNUMBER	PLOCATION	DNUM
	ProductX	1	Bellaire	5
	ProductY	2	Sugarland	5
	ProductZ	3	Houston	5
	Computerization	10	Stafford	4
	Reorganization	20	Houston	1
	Newbenefits	30	Stafford	4

DEPENDENT	ESSN	DEPENDENT_NAME	SEX	BDATE	RELATIONSHIP
	333445555	Alice	F	1988-04-05	DAUGHTER
	333445555	Theodore	M	1983-10-25	SON
	333445555	Joy	F	1959-05-03	SPOUSE
	987654321	Abner	M	1942-02-28	SPOUSE
	123456789	Michael	M	1988-01-04	SON
	123456789	Alice	F	1988-12-30	DAUGHTER
	123456789	Elizabeth	F	1967-05-05	SPOUSE

■ Query :

Make a list of all project numbers for projects that involve an employee whose last name is 'Smith', either as a worker or as a manager of the department that controls the project.

```
(SELECT DISTINCT PNUMBER
```

```
FROM PROJECT, WORKS_ON, EMPLOYEE
```

```
WHERE PNUMBER=PNO AND ESSN=SSN AND LNAME='Smith');
```

```
UNION
```

```
(SELECT DISTINCT PNUMBER FROM PROJECT, DEPARTMENT,
```

```
EMPLOYEE WHERE DNUM=DNUMBER AND MGRSSN=SSN AND
```

```
LNAME='Smith')
```

Substring Pattern Matching

- The **LIKE** comparison operator can be used for string pattern matching.
- **Partial strings are specified using two reserved characters:**
 1. % replaces an arbitrary number of zero or more characters
 2. the underscore (__) replaces a single character.

QUERY 12 : Retrieve all employees whose address is in Houston.

```
SELECT FNAME, LNAME FROM EMPLOYEE  
WHERE ADDRESS LIKE '%Houston%';
```

- Query:

Find all employees who were born during the 1950s.

```
SELECT FNAME, LNAME FROM EMPLOYEE  
WHERE BDATE LIKE '195 _ _ _ _ _ _ _ _'
```

Date format in SQL → **YYYY-MM-DD**

ARITHMETIC OPERATIONS

- The standard arithmetic operators '+', '-', '*', and '/' (for addition, subtraction, multiplication, and division, respectively) can be applied to numeric values in an SQL query result

- **Query :**

Show the resulting salaries if every employee working on the 'ProductX' project is given a 10 percent raise.

```
SELECT FNAME, LNAME, 1.1 * SALARY AS INCREASED_SAL
```

```
FROM EMPLOYEE, WORKS_ON, PROJECT
```

```
WHERE SSN=ESSN AND PNO=PNUMBER AND PNAME='ProductX';
```

BETWEEN OPERATOR:

- Retrieve all employees in department 5 whose salary is between 30,000 and 40,000.

```
SELECT * FROM EMPLOYEE
```

```
WHERE (SALARY BETWEEN 30000 AND 40000) AND DNO =5;
```

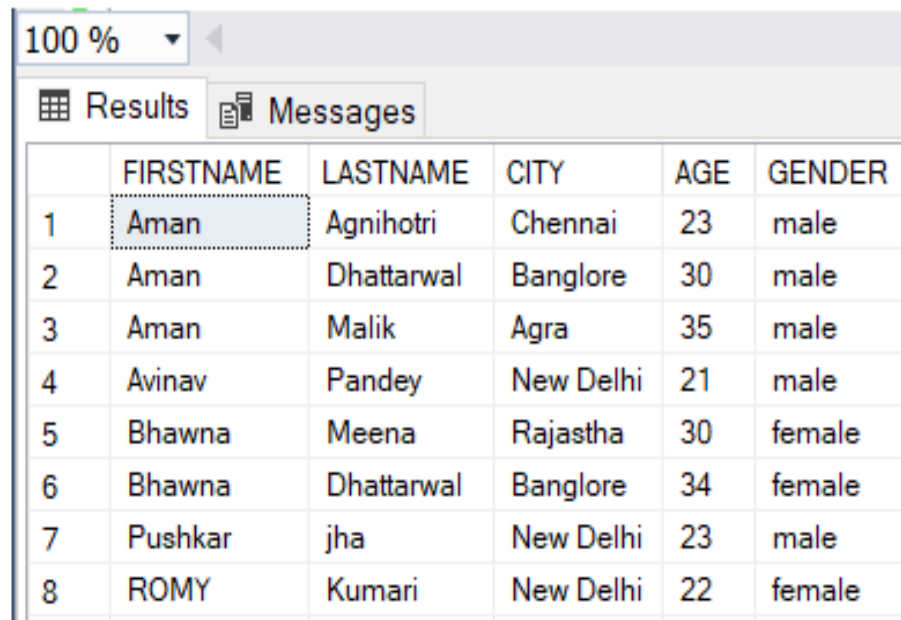
- **The condition** (Salary BETWEEN 30000 AND 40000) is equivalent to the condition ((Salary >=30000) AND (Salary <=40000))

ORDERING OF QUERY RESULTS

- The **ORDER BY** clause is used to **sort the tuples in a query result** based on the values of some attribute(s)

```
SELECT * FROM geeksforgeeks ORDER BY FIRSTNAME, AGE;
```

Output



	FIRSTNAME	LASTNAME	CITY	AGE	GENDER
1	Aman	Agnihotri	Chennai	23	male
2	Aman	Dhattarwal	Banglore	30	male
3	Aman	Malik	Agra	35	male
4	Avinav	Pandey	New Delhi	21	male
5	Bhawna	Meena	Rajastha	30	female
6	Bhawna	Dhattarwal	Banglore	34	female
7	Pushkar	jha	New Delhi	23	male
8	ROMY	Kumari	New Delhi	22	female

Query:

```
SELECT * FROM geeksforgeeks ORDER BY FIRSTNAME, LASTNAME DESC, AGE DESC ;
```

FIRSTNAME	LASTNAME	AGE
Aman	Verma	30
Aman	Sharma	25
Aman	Gupta	28
Bhawna	Mehta	24
Bhawna	Mehta	22
Bhawna	Agarwal	27

INSERT COMMAND

- INSERT is used to add a single tuple(row) to a relation(table).
- We need to specify the relation name and a list of values for the tuple.
- The values should be listed in the same order in which the corresponding attributes were specified in the CREATE TABLE command.

```
INSERT INTO EMPLOYEE VALUES ( 'John' ,B', 'Smith',  
'123456789', '1972-01-01', 'Houston, Texas', 'M', 35000,  
'888665555',4 )
```

- An alternate form of INSERT specifies explicitly the attribute names that correspond to the values in the new tuple
 - Attributes with NULL values can be left out

- **Eg:**

Insert a tuple for a new EMPLOYEE for whom we only know the FNAME, LNAME, and SSN attributes.

```
INSERT INTO EMPLOYEE (FNAME, LNAME, SSN)
VALUES ('Richard', 'Marini', '653298653')
```

- Another variation of INSERT allows insertion of *multiple tuples* resulting from a query into a relation.

DELETE

- Removes tuples from a relation. It **includes a WHERE-clause to select the tuples to be deleted**
 - Referential integrity should be enforced
- Tuples are deleted from **only one table at a time** (unless CASCADE is specified on a referential integrity constraint)
- A missing WHERE-clause specifies that **all tuples in the relation are to be deleted**; the table then becomes an empty table
- The number of tuples deleted depends on the number of tuples in the relation that satisfy the WHERE-clause

■ **Eg:**

1. DELETE FROM EMPLOYEE
WHERE LNAME='Brown'
2. DELETE FROM EMPLOYEE
WHERE SSN='123456789'
3. DELETE FROM EMPLOYEE
WHERE DNO IN
(SELECT DNUMBER
FROM DEPARTMENT
WHERE DNAME='Research')
4. DELETE FROM EMPLOYEE

UPDATE

- Used to modify attribute values of one or more selected tuples
- A WHERE-clause selects the tuples to be modified
- An additional **SET-clause** specifies the attributes to be modified and their new values
- Each command modifies tuples *in the same relation*
- Referential integrity should be enforced

■ **Eg:**

Change the location and controlling department number of project number 10 to 'Bellaire' and 5, respectively.

```
UPDATE    PROJECT
SET        PLOCATION = 'Bellaire',
           DNUM = 5
WHERE      PNUMBER=10
```

- Give all employees in the 'Research' department a 10% raise in salary.

```
UPDATE EMPLOYEE
SET     SALARY = SALARY * 1.1
WHERE   DNO IN (SELECT DNUMBER
                FROM DEPARTMENT
                WHERE DNAME='Research')
```

- In this request, the modified SALARY value depends on the original SALARY value in each tuple
 - The reference to the SALARY attribute on the right of = refers to the old SALARY value before modification
 - The reference to the SALARY attribute on the left of = refers to the new SALARY value after modification

Chapter 7

More SQL: Complex Queries, Triggers, Views and Schema Modification

NULL value and Three-Valued Logic

- **SQL has 3 kinds of NULL values.**

1. Unknown value, but existing

A particular person has a date of birth but it is not known.

2. Unavailable values :

we do not know whether a person has a home phone number.

3. Not applicable attribute value:

An attribute **CollegeDegree** would be NULL for a person who has no college degrees, because it does not apply to that person.

- SQL uses a three-valued logic with values TRUE, FALSE, and UNKNOWN
- Table 8.1 shows the result of three-valued logic when logical connectives AND, OR, and NOT are used .

TABLE 8.1 LOGICAL CONNECTIVES IN THREE-VALUED LOGIC

AND	TRUE	FALSE	UNKNOWN
TRUE	TRUE	FALSE	UNKNOWN
FALSE	FALSE	FALSE	FALSE
UNKNOWN	UNKNOWN	FALSE	UNKNOWN
OR	TRUE	FALSE	UNKNOWN
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	UNKNOWN
UNKNOWN	TRUE	UNKNOWN	UNKNOWN
NOT			
TRUE	FALSE		
FALSE	TRUE		
UNKNOWN	UNKNOWN		

- SQL allows queries that check whether an attribute value is NULL .
- Instead of using the = **or** <> to compare an attribute value to NULL, SQL uses the comparison operators **IS** or **IS NOT**.
- This is because SQL considers each NULL value distinct.
- **Eg:** Retrieve the names of all employees who do not have supervisors.

```
SELECT FNAME, LNAME FROM EMPLOYEE
```

```
WHERE SUPERSSN IS NULL;
```

Nested Query:

- A complete SELECT query, called a nested query , can be specified within the WHERE-clause of another query, called the outer query
- Many of the previous queries can be specified in an alternative form using nesting.

Syntax:

```
SELECT attribute_list  
FROM table_list  
WHERE attr_list_1 IN (  
    SELECT attr_list_1  
    FROM table_list1  
    WHERE condition  
)
```

Outer Query

Nested Query
(inner query)

- **Query 1: Make a list of all project numbers for projects that involve an employee whose last name is 'Smith' as a worker or as a manager of the department that controls the project.**

```
SELECT DISTINCT Pnumber
FROM PROJECT
WHERE Pnumber IN
    ( SELECT Pnumber
      FROM PROJECT, DEPARTMENT, EMPLOYEE
      WHERE Dnum=Dnumber AND Mgr_ssn=SSN AND
            Lname='Smith' )
```

Outer query selects **distinct project numbers**

Inner query Pnumber where last name is smith

Altogether **all distinct project numbers** managed by an employee with the last name **Smith**.

- If a nested query returns a single attribute and a single tuple, the query result is a single value.
- In such cases, we can make use of '=' instead of IN for the comparison.
- **Query 2:** Find the manager's SSN of the employee John Smith.

```
SELECT Mgr_ssn  
FROM DEPARTMENT  
WHERE DNumber = (SELECT DNo  
                  FROM EMPLOYEE  
                  WHERE FName='John' and  
                  LName='Smith'))
```

- SQL allows the use of tuples of values in comparisons by placing them within parentheses.

Query 3:

select the SSNs of all employees who work the same (project, hours) combination on some project that employee whose SSN = '123456789' works on.

SELECT DISTINCT ESSN **FROM** WORKS_ON

WHERE (PNO, HOURS) **IN**

(SELECT PNO, HOURS **FROM** WORKS_ON **WHERE** ESSN='123456789');

In this example, the IN operator compares the subtuple of values in parentheses (PNO, HOURS) for each tuple in WORKS_ON is compared with the set of union-compatible tuples produced by the nested query.

Query 4: Retrieve the name and address of all employees who work for the 'Research' department.

```
SELECT FNAME, LNAME, ADDRESS  
FROM EMPLOYEE  
WHERE DNO IN (SELECT DNUMBER  
               FROM DEPARTMENT  
               WHERE DNAME='Research' )
```

- The nested query selects the number of the 'Research' department.
- The outer query select an EMPLOYEE tuple if its DNO value is in the result of either nested query.
- The comparison operator IN compares a value v with a set (or multi-set) of values V, and evaluates to TRUE if v is one of the elements in V.
- In general, we can have several levels of nested queries.
- In this example, the nested query is not correlated with the outer query

SQL Queries and Sub Queries :

Eg:

Emp:

e_id	e_name	dept	salary
1	Ram	HR	10,000
2	Amrit	Market	20,000
3	Ravi	HR	30,000
4	Nitin	Market	40,000
5	Varun	IT	50,000

i) **Write a SQL Query to display maximum salary from Emp table**

Ans: select max(Salary) from Emp;

ii) Write a SQL Query to display Employee name who is taking maximum salary

Ans: `select e_name from emp where salary=
(select max(salary) from emp);`

iii) Write a SQL Query to display 2nd highest salary from emp table.

Ans: `select max(salary) from emp where
salary <> (select max(salary) from emp);`

Inner query finds highest salary of employee. For eg: 20,000

10,000 | 15,000 | 8,000 | 12,000 | 20,000

Outer query **excludes** the highest salary from consideration.
This effectively removes 20,000 from the list and return 2nd
highest salary

iv) Write a SQL Query to display all the department name along with number of employee working in it

Ans: select dept, count(*)
from emp
group by dept;

v) Write a SQL Query to display all the department name where number of employees are less than 2.

Ans: select dept
from emp
group by dept
having count(*) <2;

vii) Write a SQL Query to display Employee name who is working in the department whose number of employee are less than 2.

Ans: select e_name from emp
 where dept IN (select dept
 from emp
 group by dept
 having count(*) < 2);

viii) Write a query to display highest salary department wise and name of employee who is taking that salary.

Ans: select e_name from emp
 where salary IN (select max(salary)
 from emp
 group by dept);

Use of IN / NOT IN:

Eg: Emp:

eid	ename	Address
1	Ravi	Chandigarh
2	Varun	Delhi
3	Nitin	Pune
4	Robin	Bangalore
5	Ammy	Chandigarh

Project:

eid	Pid	pname	Location
1	P1	IoT	Bangalore
5	P2	Bigdata	Delhi
3	P3	Retail	Mumbai
4	P4	Android	Hyderabad

i) Find the detail of employee whose address is either Delhi or Chandigarh or Pune

Ans: select * from emp where
address IN ('Delhi', 'Chandigarh', 'Pune');

Output:

eid	ename	Address
1	Ravi	Chandigarh
2	Varun	Delhi
3	Nitin	Pune
5	Ammy	Chandigarh

NOT IN:

```
select * from emp where  
        address NOT IN ('Delhi', 'Chandigarh', 'Pune');
```

Output:

eid	ename	Address
4	Robin	Bangalore

IN/ NOT IN:

Emp:

eid	ename	Address
1	Ravi	Chandigarh
2	Varun	Delhi
3	Nitin	Pune
4	Robin	Bangalore
5	Ammy	Chandigarh

Project:

eid	Pid	pname	Location
1	P1	IoT	Bangalore
5	P2	Bigdata	Delhi
3	P3	Retail	Mumbai
4	P4	Android	Hyderabad

Find the name of the employees who are working on a project



Select ename from emp where eid IN (select eid from Project);

Output:

Ravi

Nitin

Robin

Ammy

NOT IN:

Select ename from emp where eid NOT IN (select eid from Project);

Output:

Varun

Multiple Row Sub Queries: IN, ALL, ANY, EXIST

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber
Research	5
Administration	4
Headquarters	1

Query 4: Retrieve the details of all the employee who's salary is EQUAL to average total salary based on department wise.

Select *From EMPLOYEE where salary IN (select avg(salary) from EMPLOYEE group by dno);

Query 5: Retrieve the Fname, Lname of all the employee who work for the department "Research and headquarters";

Select Fname, Lname From employee where dno IN (select Dnumber from DEPARTMENT where Dname IN (' Research', 'Headquarters'));

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

SQL allows the use of tuples (multiset) of values in comparisons by placing them in parentheses.

Query 6: Retrieve the Essn of all employee who work on the same (project, hours) combination on some project that employee 'John Smith' whose Essn is 123456789 works on.

```
SELECT DISTINCT Essn FROM WORKS_ON
WHERE (Pno, Hours) IN (SELECT Pno, Hours FROM WORKS_ON
WHERE Ssn = '123456789');
```

Query 7: Retrieve the Essn of all employee who work on the same (project, hours) combination on some project that employee 'John Smith' whose Essn is 123456789 works on.

```
SELECT Essn FROM WORKS_ON
WHERE (Pno, Hours) IN (SELECT Pno, Hours FROM WORKS_ON
WHERE Essn = '123456789');
```

THE EXISTS AND UNIQUE FUNCTIONS IN SQL

- **Exist/Not Exist** uses Top down approach i.e. compares each row of outer query with inner query table. It is used in correlated nested query. It returns True/False. If Inner query returns true then displays details from outer query. If inner query returns false then it does not display the values.
- EXISTS and UNIQUE are Boolean functions that return TRUE or FALSE.
- Hence, they can be used in a WHERE clause condition.
- The EXISTS function in SQL is used to check whether the result of a nested query is empty(contains no tuples) or not.
- The result of EXISTS is a Boolean value TRUE if the nested query result contains at least one tuple, or FALSE if the nested query result contains no tuples.

NOT EXISTS

- NOT EXISTS returns TRUE if there are no tuples in the result of the nested query and returns FALSE otherwise.
- **Query 7: Retrieve the names of employees who have no dependents.**

```
SELECT Fname, Lname  
FROM EMPLOYEE  
WHERE NOT EXISTS ( SELECT *  
                    FROM DEPENDENT  
                    WHERE SSN=ESSN )
```

Eg:
Emp:

eid	ename	Address
1	Ravi	Chandigarh
2	Varun	Delhi
3	Nitin	Pune
4	Robin	Bangalore
5	Ammy	Chandigarh

Project:

eid	Pid	pname	Location
1	P1	IoT	Bangalore
5	P2	Bigdata	Delhi
3	P3	Retail	Mumbai
4	P4	Android	Hyderabad

Find the detail of employee who is working on atleast one project

Select * from Emp where

Eid EXISTS (Select eid from Project
where emp.eid= Project.eid);

Output:

eid	ename	Address
1	Ravi	Chandigarh
3	Nitin	Pune
4	Robin	Bangalore
5	Ammy	Chandigarh

NOT EXISTS:

Select * from Emp where

Eid NOT EXISTS (Select eid from Project
where emp.eid= Project.eid);

Output:

eid	ename	Address
2	Varun	Delhi

Query 5:

Retrieve the name of each employee who has a dependent with the same first name as the employee.

```
SELECT FNAME, LNAME  
FROM EMPLOYEE  
WHERE EXISTS (SELECT *  
               FROM DEPENDENT  
               WHERE SSN=ESSN AND  
                 FNAME=DEPENDENT_NAME)
```


- **Query 6:** List the names of managers who have atleast one dependent.

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE EXISTS ( SELECT *
                FROM DEPENDENT
                WHERE SSN=ESSN )
AND
EXISTS ( SELECT *
          FROM DEPARTMENT
          WHERE SSN=MGR_SSN )
```

- **Query 7:** Retrieve the names of each employee who works on all the projects controlled by department number 5.

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE NOT EXISTS ((SELECT Pnumber
                     FROM PROJECT
                     WHERE Dnum=5)
EXCEPT
 ( SELECT Pno
   FROM WORKS_ON
   WHERE SSN=ESSN))
```

CORRELATED NESTED QUERIES

- Whenever a condition in the WHERE clause of a nested query references some attribute of a relation declared in the outer query, the two queries are said to be ***correlated***.
- We can understand a correlated query better by considering that the nested query is evaluated once for each tuple (or combination of tuples) in the outer query

Eg:

Emp:

eid	name	address
1	A	Delhi
2	B	Pune
3	A	Chandigarh
4	B	Delhi
5	C	Pune
6	D	Mumbai
7	E	Hyderabad

Dept:

Did	Dname	eid
D1	HR	1
D2	IT	2
D3	Market	3
D4	Testing	4

Find all the Employee details who works in a department

```
Select * from emp where
    EXISTS (select *
            from dept
            where dept.eid=emp.eid);
```

Output:

eid	name	address
1	A	Delhi
2	B	Pune
3	A	Chandigarh
4	B	Delhi

Eg: Retrieve the name of each employee who has a dependent with the same first name as the employee.

```
SELECT E.FNAME, E.LNAME  
FROM EMPLOYEE AS E  
WHERE E.SSN IN (SELECT ESSN  
                FROM DEPENDENT  
                WHERE ESSN=E.SSN AND  
                E.FNAME=DEPENDENT_NAME)
```

- Here the nested query has a different result *for each tuple* in the outer query.
- A query written with nested SELECT... FROM... WHERE... blocks and using the = or IN comparison operators can ***always*** be expressed as a single block query. Above example may be written as:

```
SELECT E.FNAME, E.LNAME  
FROM EMPLOYEE E, DEPENDENT D  
WHERE E.SSN=D.ESSN AND E.FNAME=D.DEPENDENT_NAME
```

Difference between Join, Nested Subquery and Correlated Subquery:

Emp:

E_id	Name
1	A
2	B
3	C
4	D
5	E

Dept:

Dept no	Name	E_id
D1	IT	1
D2	HR	2
D3	Market	3

Find the detail of all the employees who works in any department

i) Nested Subquery: Nested Subquery uses Bottom up. Executes inner query once & gives output.

**Select * from emp where eid IN (select eid
from dept);**

Output:

Eid	Name
1	A
2	B
3	C

Find the detail of all the employees who works in any department

ii) Correlated Subquery:

**Select * from emp where EXISTS (select eid
from dept
where emp.e_id=dept.e_id);**

Output:

Eid	Name
1	A
2	B
3	C

iii) Join: Cross product + Condition

select attributes from emp, dept where Emp.e_id= Dept.E_id;

Output:

Eid	Name
1	A
2	B
3	C

Eg:
Emp:

Id	Salary
1	10,000
2	20,000
3	20,000
4	30,000
5	40,000
6	50,000

Find Nth highest salary using SQL:

To find 2nd highest salary:

Select max(salary) from emp

where salary NOT IN (select max(salary)
from emp);

```
select id, salary from emp e1  
      where N-1 = ( select count(Distinct Salary)  
                from Emp e2  
                where e2.salary > e1.salary);
```

Other Comparison Operators:

- **=ANY** or **=SOME** operators returns TRUE if the value v is equal to some value in the set V and hence is equivalent to IN.
- Other operators that can be combined with SOME or ANY include >, >=, <, <= and <>.

- **Syntax:**

SELECT column_name(s)

FROM table_name

WHERE column_name operator ANY (SELECT column_name

FROM table_name

WHERE condition);

Customer:

Customer_Id	Name
1	Rahul
2	Yashpal
3	Ram
4	Virender

Reservations:

ID	Customer_Id	Date
1	1	2017-12-30
2	2	2017-12-28
3	2	2017-12-29
4	1	2017-12-25
5	3	2017-12-26

**SELECT customer_id from customers
WHERE customer_id = ANY(Select customer_id from
reservations);**

Customer_Id
1
2
3

AGGREGATE FUNCTIONS IN SQL

- Aggregate functions are used to summarize information from multiple tuples into a single tuple summary.
- Grouping is used to create subgroups of tuples before summarization.
- A number of built-in aggregate functions exists: COUNT, SUM, MAX, MIN and AVG.
- These can be used in the SELECT clause or HAVING clause.

- **Query 11:** Find the sum of the salaries of all employees, the maximum salary, the minimum salary and the average salary.

```
SELECT SUM(Salary), MAX(Salary), MIN(Salary),  
AVG(Salary)  
FROM EMPLOYEE
```

- **Renaming the attributes in the query result in the previous query**

```
SELECT SUM(Salary) as Salary_SUM, MAX(Salary) as  
MAX_salary, MIN(Salary) as MIN_Salary, AVG(Salary) as  
AVG_Salary  
FROM EMPLOYEE
```

- **Query 12:** Find the sum of the salaries of all the employees of the 'Research' department, as well as the minimum salary, maximum salary, and the average salary in the department.

```
SELECT SUM(Salary) as Salary_SUM, MAX(Salary) as  
MAX_salary, MIN(Salary) as MIN_Salary, AVG(Salary) as  
AVG_Salary  
FROM EMPLOYEE, DEPARTMENT  
WHERE Dname='Research' and DNO=Dnumber.
```


- **Query 13:** Retrieve the total number of employees in the company

```
SELECT COUNT(*)  
FROM EMPLOYEE
```

- **Query14:** Retrieve the total number of employees in the 'Research' Department

```
SELECT COUNT(*)  
FROM EMPLOYEE, DEPARTMENT  
WHERE Dname='Research' and Dno=Dnumber
```

- **Query15:** Count the number of distinct salaries.

```
SELECT COUNT(DISTINCT Salary)
FROM EMPLOYEE
```

- **Query16:** Retrieve the names of all the employees who have more than two dependents.

```
SELECT Fname, Lname  
From EMPLOYEE  
WHERE (SELECT COUNT(*)  
        FROM DEPENDENTS  
        WHERE SSN=ESSN) >=2
```

SQL Views

In SQL, views **contain rows and columns similar to a table**, however, **views don't hold the actual data**.

Creating a View in SQL

We can create views in SQL by using the `CREATE VIEW` command. For example,

```
CREATE VIEW us_customers AS
SELECT customer_id, first_name
FROM Customers
WHERE Country = 'USA';
```

Here, a view named `us_customers` is created from the `customers` table.

Now to select the customers who lives in **USA**, we can simply run,

```
SELECT *
FROM us_customers;
```



Updating a View

It's possible to change or update an existing **view** using the `CREATE OR REPLACE VIEW` command. For example,

```
CREATE OR REPLACE VIEW us_customers AS
SELECT *
FROM Customers
WHERE Country = 'USA';
```

Deleting a View

We can delete views using the `DROP VIEW` command. For example,

```
DROP VIEW us_customers;
```



1 VIEW (Stored Query)

- A **VIEW** is a **virtual table** stored in the database.
- You can **reuse** it in multiple queries without writing the same logic again.
- It persists in the database until explicitly dropped.

Advantages of VIEW:  Stored in the database and reusable.

 Can improve security (restrict access to columns).

 Can be indexed for better performance.



Triggers in SQL

- A **trigger** is a rule in a database that runs automatically **when** data is inserted, updated, or deleted.
- It defines when it should run and **what action** it should perform.
- Triggers are mainly used to maintain rules and relationships in the database that cannot be handled using normal SQL constraints.



- **Eg:** When a new record is added to the **Donors** table (someone donates blood), a **trigger** automatically adds that information to the **Blood_record** table. So, no need to manually update both tables—the trigger does it for you.

Types of Triggers:

- **AFTER INSERT** : Runs after new data is added.
- **AFTER UPDATE** : Runs after data is changed.
- **AFTER DELETE** : Runs after data is removed.
- **BEFORE INSERT** : Runs before new data is added.
- **BEFORE UPDATE** : Runs before data is changed.
- **BEFORE DELETE** : Runs before data is removed.

BEFORE → happens before the action

AFTER → happens after the action



1. Write a trigger to ensure that no employee of age less than 25 can be inserted in the database.

```
delimiter $$
CREATE TRIGGER Check_age BEFORE INSERT ON employee
FOR EACH ROW
BEGIN
IF NEW.age < 25 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'ERROR:
                AGE MUST BE ATLEAST 25 YEARS!';
END IF;
END; $$
delimiter;
```

delimiter \$\$: It tells MySQL to treat \$\$ as the end of the trigger definition.

```
CREATE TRIGGER Check_age BEFORE INSERT ON employee
```

- Creates a trigger named **Check_age**.
- **BEFORE INSERT** means it runs **before a new row is added**.
- **ON employee** means it works on the **employee table**.



FOR EACH ROW

- The trigger runs **once for every row** being inserted.
 - If multiple rows are inserted, it checks each one.
-

BEGIN

- Marks the **start** of the trigger body.
- All logic goes inside this block.

IF NEW.age < 25 THEN

- **NEW** refers to the new record being inserted.
 - **NEW.age** means the age value of that new employee.
 - This line checks: **Is the age less than 25?**
-

SIGNAL SQLSTATE '45000'

- Stops the insertion process.
- **'45000'** is a general error code in MySQL.
- It tells MySQL to raise a custom error.

```
SET MESSAGE_TEXT = 'ERROR: AGE MUST BE ATLEAST 25 YEARS!';
```

- Displays a custom error message.
 - The user will see this message if age is less than 25.
-

```
END IF;
```

- Ends the IF condition.
-

```
END; $$
```

- Ends the trigger body.
 - `$$` marks the end of the full trigger definition.
-

```
delimiter ;
```

- Changes the delimiter back to the default `;`.

- Whenever we want to insert any tuple to table 'employee', then before inserting this tuple to the table, trigger named '**Check_age**' will be executed.
- This trigger will check the age attribute. If it is greater than 25 then this tuple will be inserted into the table otherwise an error message will be printed stating "ERROR: AGE MUST BE ATLEAST 25 YEARS!"
- **45,000 is a custom SQLSTATE error code** used to signal an exception in MySQL. **SQLSTATE** is a 5-character alphanumeric code that represents a specific SQL error.
- 45000 is a generic user-defined error code, that means "unhandled user-defined exception."



A typical trigger has three main components:

1. **Event** – This is the database operation that activates the trigger, such as **INSERT**, **UPDATE**, or **DELETE**.
2. **Condition (optional)** – This checks whether the action should run. After the event happens, the condition is evaluated. If it is true, the action is executed. It is written in the **WHEN clause**.
3. **Action** – This is the set of **SQL statements** that run when the event occurs (and the condition is true, if given).



- Consider the following schema for College database:
- **STUDENT(USN, SName, Address, Phone, Gender)**
- **SEMSEC(SSID, Sem, Sec);**
- **CLASS(USN, SSID)**
- **SUBJECT(Subcode, Title, Sem, Credits)**
- **MSEMARKS(USN, Subcode, SSID, MSE1, MSE2, Task, TotalMarks)**

Create a trigger on MSE MARKS table such that total marks is automatically calculated and inserted whenever a new record is inserted to the table.

```
CREATE TRIGGER TOTAL
BEFORE INSERT
ON MSEMARKS
FOR EACH ROW
SET NEW.TOTALMARKS = NEW.MSE1 + NEW.MSE2 +
NEW.TASK;
```



ASSERTION

An ASSERTION in SQL is a rule that makes sure a **condition in the database is always true**. It is used to maintain database integrity. If the condition becomes **false**, it means there is an **error or problem in the database**.

Creating an Assertion

Here's how to create an assertion in SQL:

```
CREATE ASSERTION assertion_name  
CHECK (condition);
```

To drop an assertion, you can use the following statement:

```
DROP ASSERTION assertion_name;
```



Example of an Assertion

Let's consider a simple example. Suppose we have a table "Orders" and we want to ensure that the total amount of all orders is always less than 10000.

```
CREATE ASSERTION orders_total  
CHECK ((SELECT SUM(amount) FROM Orders) < 10000);
```

This assertion ensures that the total amount of all orders will never exceed 10000. If an attempt is made to add an order that would cause the total to exceed 10000, the database system will reject it.



```
CREATE ASSERTION SALARY_CONSTRAINT
CHECK ( NOT EXISTS ( SELECT *
                     FROM   EMPLOYEE E, EMPLOYEE M,
                     DEPARTMENT D
                     WHERE  E.Salary > M.Salary
                     AND     E.Dno = D.Dnumber
                     AND     D.Mgr_ssn = M.Ssn ) );
```

- By including this query inside a **NOT EXISTS clause**, the assertion will specify that **the result of this query must be empty** so that the condition will always be **true**.
- The **assertion is violated** if the result of the query is **not empty**.



SCHEMA CHANGE STATEMENTS IN SQL

- DROP command
 - Used to drop named schema elements, such as tables, domains, or constraint
- Drop behavior options:
 - CASCADE and RESTRICT
- Example:
 - DROP SCHEMA COMPANY CASCADE;
 - This removes the schema and all its elements including tables, views, constraints, etc.



CASCADE and RESTRICT :

- These are the options used with FOREIGN KEY constraints to control what happens to child records when a parent record is updated or deleted.
- They are commonly used with:
ON DELETE
ON UPDATE

✓ What is CASCADE?

CASCADE means:

When a parent record is **deleted or updated**, the related child records are **automatically deleted or updated** as well.



Eg: Create Parent Table:

```
CREATE TABLE departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);
```

Create Child Table with CASCADE

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    dept_id INT,  
    FOREIGN KEY (dept_id)  
    REFERENCES departments(dept_id)  
    ON DELETE CASCADE  
);
```



Insert Data:

```
INSERT INTO departments VALUES (1, 'HR');  
INSERT INTO employees VALUES (101, 'John', 1);  
INSERT INTO employees VALUES (102, 'Alice', 1);
```

Delete Parent Record:

```
DELETE FROM departments WHERE dept_id = 1;
```

✓ Result:

- Department 1 is deleted
- Employees John and Alice are automatically deleted

Because of `ON DELETE CASCADE`.



ON UPDATE CASCADE Example

```
FOREIGN KEY (dept_id)  
REFERENCES departments(dept_id)  
ON UPDATE CASCADE
```

If dept_id changes in departments, it automatically updates in employees.

What is RESTRICT?

RESTRICT means:

You cannot delete or update a parent record if child records exist.

It prevents accidental data loss.

Example: ON DELETE RESTRICT

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    dept_id INT,  
    FOREIGN KEY (dept_id)  
    REFERENCES departments(dept_id)  
    ON DELETE RESTRICT  
);
```

If you run: DELETE FROM departments WHERE dept_id = 1;

Result:

You get an error like:

</> Code

```
Cannot delete or update a parent row: a foreign key constraint fails
```

Because employees still reference that department.

2.The ALTER Command

▪The definition of a base table or of other named schema elements can be changed by using the ALTER command.

- **Alter table actions** include:
 - Adding or dropping a column (attribute)
 - Changing a column definition
 - Adding or dropping table constraints

▪ **Eg:** To add an attribute for keeping track of jobs of employees to the EMPLOYEE base relation in the COMPANY schema, we can use the command

```
ALTER TABLE COMPANY.EMPLOYEE ADD COLUMN  
Job VARCHAR(12);
```



- It is also possible to alter a column definition by dropping an existing default clause or by defining a new default clause.

```
ALTER TABLE COMPANY.DEPARTMENT  
ALTER COLUMN Mgr_ssn DROP DEFAULT;
```

```
ALTER TABLE COMPANY.DEPARTMENT  
ALTER COLUMN Mgr_ssn SET DEFAULT '333445555';
```



- One can also change the constraints specified on a table by adding or dropping a named constraint. To be dropped, a constraint must have been given a name when it was specified.
- **Eg:** To drop the constraint named fk2 from the EMPLOYEE relation, we write:

```
ALTER TABLE COMPANY.EMPLOYEE
```

```
DROP CONSTRAINT fk2
```

